

República Bolivariana de Venezuela  
Ministerio del Poder Popular para la Educación  
Guanta, Edo – Anzoátegui  
Ingeniería del Software



# INFORME

Alumno: Jesús Acosta V-29.733.586

Profesor: Hummer Lenis

## Introducción al Sistema

- Desde hace ya bastante tiempo, el hombre ha tenido la necesidad de procesar y transmitir la información de una forma continuada. Para conseguir los objetivos deseados, se han utilizado diferentes técnicas y medios. El término Informática ha evolucionado a lo largo del tiempo. Al principio, se definió como la ciencia que se encargaba de estudiar el tratamiento automático de la información. Procede de la concentración de dos palabras: información y automática. Tradicionalmente se han utilizado medios como el correo convencional, el teléfono, etc., pero hoy en día los medios y formas de transmitir información han cambiado considerablemente. El tratar y transmitir cada vez más información de diferente tipo y de forma continuada y sistemática por parte del ser humano, ha contribuido a que poco a poco se vayan creando herramientas que faciliten una solución a estas necesidades. Hace algunos años se empezaron a diseñar y construir herramientas complejas para procesar y transmitir información. Estas herramientas se han podido desarrollar gracias al gran avance tecnológico iniciado a partir de los años ochenta. El ordenador es la herramienta que permite procesar automáticamente la información, facilitando en gran medida su organización, tratamiento, transmisión y almacenamiento.

- El término Informática ha evolucionado a lo largo del tiempo. Al principio, se definió como la ciencia que se encargaba de estudiar el tratamiento automático de la información. Procede de la concentración de dos palabras: información y automática.

- En la actualidad, esta palabra engloba muchos más conceptos, ya que se aplica a la mayoría de los campos técnicos y científicos. La informática, hoy por hoy, se utiliza en la mayoría de los campos y actividades en las que el ser humano es partícipe. Se utiliza en medicina, aviación, construcción, diseño de automóviles, edificios, etc. y, por supuesto, se utiliza de forma personal para cubrir necesidades que hace unos años no existían. Mediante la Informática se gestiona información en forma de programas y datos con los que es posible poner en marcha complejos sistemas electrónicos, mecánicos, etc. como, por ejemplo, controlar maquinaria, orientar y poner en órbita naves espaciales, controlar los ciclos de producción de cosechas, etc. Su desarrollo ha sido espectacular en las dos últimas décadas del siglo XX, convirtiéndose en una herramienta imprescindible en comunicaciones, telefonía, Internet, vigilancia, control de tráfico, etc.

## Análisis de un Sistema

- El análisis de un sistema informático consiste en estudiar una situación, un problema o un proceso de negocio para definir sus componentes y proponer mejoras mediante la implementación de software. Se basa en la descomposición de un sistema complejo en sus partes elementales para comprender su estructura, su comportamiento y sus interacciones.

### **- Objetivos principales del análisis:**

- **Identificación de necesidades:** Comprender los problemas del sistema actual (manual o digitalizado de forma ineficiente).
- **Levantamiento de requerimientos:** Definir con exactitud qué funciones debe cumplir el nuevo software para resolver el problema.
- **Definición de fronteras:** Establecer los límites del sistema (qué procesos abarcará y cuáles quedarán fuera de su alcance).
- **Estudio de viabilidad:** Determinar si el desarrollo del sistema es técnica, económica y operativamente posible.

### **Actividades clave en esta fase:**

1. **Recolección de datos:** Uso de entrevistas, cuestionarios, observación directa y revisión de documentos técnicos o administrativos.
2. **Modelado del sistema:** Creación de representaciones visuales de los procesos actuales y futuros utilizando herramientas lógicas como Diagramas de Flujo de Datos (DFD) o diagramas de Casos de Uso (UML).
3. **Especificación de Requisitos del Software (ERS):** Redacción de un documento formal que detalla los requisitos funcionales (lo que el sistema debe hacer) y no funcionales (rendimiento, seguridad, usabilidad).

### **- Análisis de un Sistema Real: Caso Práctico (Biblioteca)**

Para ilustrar la teoría, a continuación, se presenta el análisis aplicado a un **Sistema de Gestión de Biblioteca (SGB)**.

**A. Planteamiento del Problema (Estado Actual)** Una biblioteca local gestiona su inventario y los préstamos de libros mediante fichas de cartón y libros de registro manuales. Esto genera redundancia de datos, pérdida ocasional del historial de préstamos, dificultad para auditar el inventario y tiempos de espera prolongados para los usuarios.

**B. Identificación de Actores del Sistema** Los actores son las entidades (personas u otros sistemas) que interactuarán directamente con el software:

- **Bibliotecario (Administrativo):** Encargado de registrar libros, aprobar préstamos y gestionar devoluciones.
- **Usuario (Lector):** Persona registrada que busca libros y solicita préstamos.
- **Administrador del Sistema:** Personal técnico de soporte, encargado de respaldos de bases de datos y gestión de permisos.

**C. Análisis de Procesos Centrales (Casos de Uso)** El sistema debe digitalizar y optimizar los siguientes procesos lógicos:

- **Gestión de Inventario:** Altas, bajas y modificaciones de los ejemplares (libros, revistas).
- **Gestión de Usuarios:** Registro de nuevos lectores y actualización de su estado (solvente o penalizado).
- **Proceso de Préstamo:** Verificación de disponibilidad del ejemplar, validación del estado del usuario y registro de la fecha de salida y retorno esperado.
- **Proceso de Devolución:** Registro de entrada del ejemplar y cálculo automático de multas en caso de retraso.

#### **D. Definición de Requerimientos**

##### **Requerimientos Funcionales (Comportamiento del sistema):**

- El sistema debe permitir la búsqueda de libros por título, autor, género o código ISBN.
- El sistema debe bloquear automáticamente la opción de préstamo a usuarios que tengan multas pendientes o libros vencidos.
- El sistema debe generar un reporte diario, en formato PDF o Excel, con los préstamos realizados en la jornada.

##### **Requerimientos No Funcionales (Atributos de calidad del sistema):**

- **Rendimiento:** Las consultas al catálogo de libros deben resolverse en un tiempo no mayor a 2 segundos.
- **Seguridad:** El acceso al módulo administrativo requerirá autenticación mediante usuario y contraseña encriptada.
- **Disponibilidad:** El sistema debe operar en una arquitectura de base de datos relacional con respaldos automáticos diarios a las 00:00 horas.

### Organización como Sistema

Para la Ingeniería del Software, una organización no es solo un conjunto de personas o departamentos, sino un sistema abierto y dinámico. Bajo el enfoque sistémico, una organización se define como una red compleja de elementos interrelacionados (recursos humanos, tecnológicos, financieros y procedimentales) que interactúan constantemente para transformar insumos en resultados, persiguiendo un objetivo común.

#### **Propiedades de la Organización como Sistema:**

- **Interdependencia:** Los cambios en un módulo (ej. contabilidad) afectan inevitablemente a otros (ej. ventas o inventario).
- **Sinergia:** El funcionamiento conjunto de todos los subsistemas produce un resultado superior a la suma de sus partes aisladas.
- **Fronteras y Entorno:** La organización interactúa con agentes externos (proveedores, clientes, entes reguladores) que le proporcionan entradas (inputs) y reciben sus salidas (outputs).
- **Retroalimentación (Feedback):** Mecanismos de control que permiten a la organización evaluar sus salidas y ajustar sus procesos internos para corregir desviaciones.

#### **Mapeo de Procesos y Flujos de Información (Caso: Sistema de Biblioteca)**

El mapeo de procesos es la representación lógica y secuencial de cómo la información se mueve a través de los distintos subsistemas de la organización elegida (la Biblioteca). Esto permite a los analistas identificar cuellos de botella, redundancias y puntos críticos antes de desarrollar el software.

A continuación, se mapea el proceso central de la organización: El Préstamo de Material Bibliográfico.

## **1. Definición de Entidades y Subsistemas**

- **Entidades Externas:** Usuario (Lector).
- **Subsistemas Internos:** Módulo de Usuarios, Módulo de Inventario (Catálogo), Módulo de Transacciones (Préstamos).

## **2. Flujo de Información Secuencial (Paso a Paso)**

El siguiente flujo detalla el movimiento de los datos desde que se inicia la solicitud hasta que se completa la transacción:

1. **Entrada de Datos (Input):** \* El Usuario proporciona sus credenciales (ej. número de carnet) y el código de identificación del libro solicitado.

2. **Consulta y Validación (Procesamiento Lógico):**

El sistema consulta el Módulo de Usuarios para verificar el estado del lector (¿Está activo? ¿Tiene multas pendientes? ¿Ha excedido el límite de préstamos?).

Si la validación de usuario es negativa: El flujo se interrumpe y se emite un mensaje de alerta (Salida).

Si la validación de usuario es positiva: El sistema consulta el Módulo de Inventario para verificar la disponibilidad física del ejemplar.

3. **Ejecución de la Transacción:**

Si el libro está disponible, el sistema vincula lógicamente el ID del Usuario con el ID del Libro.

Se genera una marca de tiempo (timestamp) con la fecha de salida y se calcula automáticamente la fecha límite de devolución.

4. **Actualización de Estados (Retroalimentación Interna):**

El Módulo de Inventario cambia el estado del libro de "Disponible" a "Prestado".

El historial del Módulo de Usuarios se actualiza con la nueva transacción.

5. **Salida de Datos (Output):**

El sistema genera un comprobante digital o físico con los detalles del préstamo para el usuario.

### 3. Análisis de la Naturaleza del Flujo

En este mapeo se evidencian dos tipos de flujos de información críticos para el diseño de la arquitectura del software:

- **Flujo Horizontal (Operativo):** Es la comunicación directa entre los módulos de base de datos (Inventario - Usuarios) para completar la transacción diaria. Es un flujo rápido y de alto volumen.
- **Flujo Vertical (Gerencial):** Es la información consolidada que el sistema envía hacia la administración de la biblioteca (ej. reportes mensuales de "libros más leídos" o "índice de morosidad"). Esta información procesada permite la toma de decisiones estratégicas, demostrando el ciclo de retroalimentación de la organización como sistema.

## Fundamentos de Ingeniería del Software

La Ingeniería del Software aplica un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento del software. Dos de los pilares metodológicos fundamentales para garantizar el éxito de un proyecto informático son la correcta gestión de los requisitos y la definición de un ciclo de vida estructurado.

### 1. Clasificación de Requisitos del Software

En el ámbito de la ingeniería de sistemas, un requisito se define como una condición o capacidad que un sistema debe poseer para satisfacer un contrato, estándar, especificación u otro documento formalmente impuesto. La Ingeniería de Requisitos clasifica estas necesidades en tres categorías principales para evitar ambigüedades durante la fase de diseño técnico:

- **Requisitos Funcionales (RF):** Describen explícitamente las interacciones entre el sistema y su entorno (usuarios u otros sistemas). Definen qué debe hacer el software, detallando los procesos de entrada, transformación de datos y salida.

Ejemplo técnico: "El módulo de autenticación debe validar las credenciales del usuario contra la base de datos cifrada antes de conceder acceso al panel de control".

- **Requisitos No Funcionales (RNF):** Establecen las restricciones y atributos de calidad bajo los cuales debe operar el sistema. No se centran en las tareas específicas del usuario, sino en el cómo el sistema ejecuta dichas tareas. Se subdividen comúnmente utilizando modelos como FURPS (Funcionalidad, Usabilidad, Confiabilidad, Rendimiento y Soporte).

Ejemplo técnico: "El sistema debe garantizar un tiempo de respuesta máximo de 1.5 segundos para consultas simultáneas de hasta 500 usuarios" (Rendimiento).

Ejemplo técnico: "La arquitectura de la base de datos debe realizar respaldos automatizados diarios a las 00:00 horas" (Confiabilidad).

- **Requisitos de Dominio (o Restricciones del Negocio):** Son aquellos derivados directamente del marco legal, normativo o del entorno operativo específico de la organización, independientemente de las necesidades de un usuario particular.

Ejemplo técnico: "El sistema de facturación debe generar los comprobantes electrónicos bajo los estándares XML exigidos por la autoridad tributaria nacional".

## 2. Ciclo de Vida del Desarrollo de Software (SDLC)

El Ciclo de Vida del Software (SDLC, por sus siglas en inglés) es el marco de referencia que detalla las fases necesarias para concebir, desarrollar, desplegar y mantener un producto de software. Su objetivo es minimizar los riesgos técnicos y asegurar la calidad del producto final mediante una ejecución controlada.

Independientemente del paradigma metodológico (predictivo como Cascada, o adaptativo como Scrum), el ciclo de vida estándar se compone de las siguientes fases secuenciales y lógicas:

1. **Análisis y Especificación de Requisitos:** Fase inicial donde se realiza el levantamiento de información. Se definen las fronteras del sistema, la viabilidad técnica y operativa, y se redacta el Documento de Especificación de Requisitos del Software (ERS).
2. **Diseño de la Arquitectura y del Sistema:** Traducción de los requisitos abstractos en representaciones técnicas. Incluye el modelado de bases de datos (diagramas Entidad-Relación), diseño de interfaces (UI/UX) y la definición de la arquitectura de red y software (ej. patrón Modelo-Vista-Controlador o Microservicios).

3. **Implementación (Codificación):** Fase de construcción donde los ingenieros traducen los modelos de diseño a código fuente ejecutable, utilizando los lenguajes de programación, librerías y frameworks predefinidos en la arquitectura.
4. **Pruebas (Testing y Aseguramiento de Calidad - QA):** Ejecución de protocolos de validación para identificar vulnerabilidades y defectos lógicos (bugs). Comprende pruebas unitarias, de integración, pruebas de estrés y pruebas de aceptación del usuario (UAT) para garantizar que el software cumple con la ERS.
5. **Despliegue (Deployment):** Transición del software desde el entorno de desarrollo hacia el entorno de producción. Implica la configuración de servidores, migración de datos y la liberación del sistema para los usuarios finales.
6. **Operación y Mantenimiento:** Fase prolongada que inicia post-despliegue. Incluye la monitorización del sistema, la aplicación de parches de seguridad (mantenimiento correctivo), la optimización de algoritmos (mantenimiento preventivo) y la adición de nuevos módulos (mantenimiento evolutivo).

### Paradigmas de Programación

La comparación entre el **Paradigma Estructurado** y la **Programación Orientada a Objetos (POO)** representa uno de los debates arquitectónicos más importantes en la evolución de la ciencia de la computación. Ambos paradigmas ofrecen filosofías radicalmente opuestas sobre cómo modelar la realidad, organizar el código fuente y gestionar el flujo de datos dentro de un sistema de información.

#### **El Paradigma Estructurado: Enfoque Procedimental**

La programación estructurada, consolidada en la década de 1970 a partir del teorema de Böhm-Jacopini, se fundamenta en la **descomposición funcional**. Bajo este enfoque, el problema principal se divide jerárquicamente en subproblemas más pequeños a través de funciones, procedimientos o subrutinas. La lógica del programa es lineal y predictiva, gobernada estrictamente por tres estructuras de control básicas: la secuencia (ejecución paso a paso), la selección (condicionales if/else) y la iteración (bucles for/while).

En este modelo, **los datos y las funciones que los procesan están completamente separados**. Los datos suelen almacenarse en estructuras pasivas (como registros o variables globales) y las funciones actúan como agentes externos que reciben estos datos, los transforman y devuelven un resultado.

- **Limitación Crítica:** A medida que el sistema crece, el volumen de funciones que acceden y modifican las variables globales se vuelve inmanejable. Un cambio menor en la estructura de una variable central puede desencadenar un efecto dominó de errores lógicos en múltiples procedimientos independientes, un fenómeno conocido como **alto acoplamiento**.

### **El Paradigma Orientado a Objetos (POO): Enfoque de Entidades**

La Programación Orientada a Objetos surge para mitigar las limitaciones de la escala estructural, cambiando el foco de atención: **en lugar de centrarse en las acciones (funciones), se centra en los elementos que reciben las acciones (datos/entidades)**. Un programa en POO se concibe como una red de objetos independientes que cooperan entre sí mediante el envío de mensajes. Un objeto es una entidad autocontenida que fusiona de manera indisoluble el estado (datos almacenados en atributos) y el comportamiento (operaciones lógicas definidas en métodos).

La arquitectura de la POO se sostiene sobre cuatro pilares fundamentales que garantizan la robustez del código:

1. **Abstracción:** Permite modelar los elementos del mundo real seleccionando únicamente las propiedades esenciales para el negocio e ignorando los detalles técnicos irrelevantes.
2. **Encapsulamiento:** Reúne los datos y métodos en una misma estructura y restringe el acceso directo a los atributos desde el exterior. El estado de un objeto solo puede ser alterado de forma segura por sus propios métodos (interfaces públicas), protegiendo la integridad de los datos.
3. **Herencia:** Facilita la creación de jerarquías donde las clases hijas heredan automáticamente las propiedades de las clases padres, eliminando la duplicación de código y promoviendo la reutilización masiva.
4. **Polimorfismo:** Permite que una misma firma de método se comporte de manera diferente según el objeto que lo ejecute, ofreciendo una enorme flexibilidad para extender el software sin modificar la lógica existente.

### **Diferencias Clave e Impacto en la Ingeniería**

- **Estructura del Código:** La programación estructurada organiza el software con una estructura de "arriba hacia abajo" (Top-Down), mientras que la POO adopta un diseño "de abajo hacia arriba" (Bottom-Up), construyendo componentes de software reutilizables y autónomos que luego se integran.
- **Mantenibilidad y Escalabilidad:** En sistemas complejos, la POO ofrece una superioridad clara gracias al **bajo acoplamiento y alta cohesión**. Si se requiere modificar el comportamiento de una entidad, los cambios se aíslan dentro de su clase correspondiente sin afectar al resto de la aplicación, reduciendo drásticamente la aparición de bugs colaterales y la acumulación de deuda técnica.

### Calidad de Software

El análisis de la calidad del software es una fase fundamental en la ingeniería, ya que determina la viabilidad, sostenibilidad y robustez de un sistema en entornos de producción. Según estándares internacionales como la familia de normas **ISO/IEC 25010 (SQuaRE)**, la calidad no es un concepto abstracto, sino un conjunto de características medibles y cuantificables.

#### **1. Mantenibilidad (Maintainability)**

La mantenibilidad representa el grado de eficacia y eficiencia con el que un sistema informático o componente puede ser modificado por los ingenieros de software después de su despliegue inicial. Estas modificaciones tienen como objetivo corregir fallos, mejorar el rendimiento o adaptar el sistema a un entorno tecnológico cambiante.

Un software altamente mantenible reduce drásticamente los costos a largo plazo y evita la acumulación de la llamada "deuda técnica". Se evalúa a través de los siguientes subatributos:

- **Modularidad:** El sistema está compuesto por componentes independientes. Un cambio en un módulo tiene un impacto mínimo en los demás (bajo acoplamiento).
- **Analizabilidad:** Facilidad con la que se puede auditar el código para identificar el origen de un fallo o evaluar el impacto de un cambio propuesto.

- **Modificabilidad:** Capacidad del código para ser alterado sin introducir nuevos defectos lógicos, lo cual se logra mediante el uso de patrones de diseño limpios y el encapsulamiento.
- **Capacidad de Prueba (Testability):** Facilidad para establecer criterios de prueba y ejecutar comprobaciones automatizadas que validen que las modificaciones no han roto funciones preexistentes.

## 2. Confiabilidad (Reliability)

La confiabilidad (o fiabilidad) es la capacidad que tiene el software para mantener su nivel de ejecución bajo condiciones específicas durante un período de tiempo determinado. Es un atributo crítico, especialmente en sistemas donde un fallo puede resultar en pérdidas financieras, interrupción del servicio o riesgos operativos.

Un sistema confiable no asume que el entorno será perfecto; por el contrario, está diseñado para anticipar y manejar errores. Sus dimensiones técnicas son:

- **Madurez:** El grado en que el sistema ha superado sus fases de depuración y cumple con las necesidades de fiabilidad bajo operación normal.
- **Tolerancia a Fallos:** La capacidad del software para seguir operando, incluso si se presentan errores lógicos en ciertos módulos, fallos en el hardware o caídas en la red (ej. mediante el manejo de excepciones o balanceo de cargas).
- **Capacidad de Recuperación (Recoverability):** Si ocurre una interrupción catastrófica, es el tiempo y esfuerzo que requiere el sistema para restablecer su nivel de operación normal y recuperar los datos afectados, garantizando su integridad estructural.

## 3. Eficiencia de Desempeño (Performance Efficiency)

La eficiencia evalúa la relación entre el rendimiento del software y la cantidad de recursos que consume, asumiendo que opera bajo las condiciones establecidas. Un código puede ser funcionalmente correcto, pero si agota la memoria del servidor o tarda demasiado en responder, carece de calidad.

Este atributo es fundamental para garantizar la escalabilidad del sistema informático frente a un aumento en la demanda de los usuarios. Se divide en:

- **Comportamiento Temporal:** Los tiempos de respuesta, tiempos de procesamiento y tasas de transferencia del sistema al realizar sus funciones (ej. resolver una consulta a la base de datos en menos de 1 segundo).

- **Utilización de Recursos:** La cantidad de recursos de hardware (CPU, memoria RAM, espacio en disco, ancho de banda) consumidos durante la ejecución de sus procesos. La optimización algorítmica es clave en este punto.
- **Capacidad:** El límite máximo de parámetros que el sistema puede manejar antes de degradarse (ej. número máximo de conexiones concurrentes o volumen de transacciones por minuto).

### Metodologías de Desarrollo

La elección de la metodología de desarrollo define la estructura, la planificación y el control del ciclo de vida de un proyecto de software. Históricamente, la ingeniería ha transitado desde enfoques estrictamente predictivos y documentales hacia modelos iterativos y adaptables.

#### **1. Modelos Tradicionales (Enfoque Predictivo)**

Los modelos tradicionales, cuyo máximo exponente es el **Modelo en Cascada (Waterfall)** y arquitecturas más robustas como el **Proceso Unificado (UP/RUP)**, se basan en un ciclo de vida lineal y secuencial. Se asume que los requisitos del sistema pueden (y deben) definirse de manera exhaustiva y precisa antes de escribir una sola línea de código.

- **Características Principales:**
  - **Fases Rígidas:** El flujo avanza unidireccionalmente: Análisis - Diseño - Implementación - Pruebas - Despliegue. Una fase no comienza hasta que la anterior ha sido completada, documentada y formalmente aprobada.
  - **Alta Carga Documental:** El conocimiento del proyecto reside en especificaciones formales (como el documento ERS o diagramas UML exhaustivos) más que en la comunicación verbal.
  - **Control de Cambios:** Cualquier modificación a los requisitos una vez iniciada la fase de diseño o codificación requiere un proceso burocrático estricto (Comité de Control de Cambios), lo que encarece la adaptación.

- **Casos de Uso Ideales:** Sistemas de misión crítica (ej. software aeroespacial, dispositivos médicos), proyectos con normativas gubernamentales estrictas, o contratos de precio y alcance cerrado donde la volatilidad de los requisitos es cercana a cero.

## 2. Modelos Ágiles (Enfoque Adaptativo)

Las metodologías ágiles (como **Scrum, Kanban o Extreme Programming - XP**) nacen como respuesta a las altas tasas de fracaso de los proyectos tradicionales frente a entornos de negocio dinámicos. Se basan en los valores del Manifiesto Ágil (2001), priorizando la entrega de valor temprano, la colaboración con el cliente y la respuesta al cambio.

- **Características Principales:**

**Desarrollo Iterativo e Incremental:** El proyecto se divide en ciclos de trabajo cortos (llamados Sprints en Scrum, típicamente de 2 a 4 semanas). Al final de cada ciclo, se entrega un incremento de software funcional y potencialmente desplegable.

**Requisitos Evolutivos:** En lugar de un documento estático, los requisitos se gestionan en una lista priorizada y dinámica (el Product Backlog). El alcance es flexible y se ajusta según la retroalimentación del mercado.

**Equipos Auto-organizados:** Menos jerarquía gerencial y mayor comunicación directa (reuniones diarias de sincronización).

- **Casos de Uso Ideales:** Startups, desarrollo de aplicaciones móviles/web, innovación tecnológica, y cualquier proyecto donde el cliente no tenga claro el producto final desde el día uno.

### Modelado de Sistemas (UML)

El **Lenguaje de Modelado Unificado (UML)** es el estándar internacional para visualizar, especificar, construir y documentar los artefactos de un sistema de software. Su propósito principal es cerrar la brecha entre los requisitos de negocio y la implementación técnica, proporcionando un lenguaje visual común para todo el equipo de desarrollo.

## 1. Diagrama de Casos de Uso

El diagrama de casos de uso se utiliza para definir el **alcance funcional** del sistema. No describe cómo funciona internamente, sino qué servicios ofrece y quiénes (actores) interactúan con él.

- **Componentes:**

**Actores:** Personas o sistemas externos que interactúan con el software.

**Casos de Uso:** Funciones principales del sistema (representadas por elipses).

## 2. Diagrama de Actividades

El diagrama de actividades es una representación gráfica de la **lógica procedimental** o el flujo de trabajo (workflow) dentro del sistema. Se asemeja a un diagrama de flujo avanzado, pero con capacidad para modelar concurrencia.

- **Componentes:**

**Nodos de acción:** Pasos o tareas específicas del proceso.

**Nodos de decisión:** Puntos de ramificación (rombos) que permiten caminos alternativos.

**Barras de sincronización:** Permiten modelar procesos que ocurren en paralelo (concurrencia).

## 3. Diagrama de Clases

Es el corazón de la POO. El diagrama de clases es una representación **estática y estructural** que describe la arquitectura interna del sistema mediante la identificación de los objetos que lo componen, sus datos y sus relaciones.

- **Componentes:**

**Clases:** Representadas por rectángulos divididos en tres secciones: Nombre, Atributos (datos) y Métodos (operaciones).

**Relaciones:**

**Asociación:** Relación básica entre dos clases.

**Herencia (Generalización):** Clase hija que hereda de una clase padre.

**Composición/Agregación:** Relaciones de "pertenencia" (todo-parte).

## Herramientas CASE

Las herramientas CASE (**Computer-Aided Software Engineering** o Ingeniería del Software Asistida por Computadora) constituyen un conjunto de aplicaciones informáticas destinadas a automatizar y optimizar las diversas etapas del ciclo de vida del desarrollo de software (SDLC)

### **1. Definición y Alcance de las Herramientas CASE**

Las herramientas CASE actúan como un entorno de trabajo integrado donde los ingenieros pueden crear, editar, validar y gestionar los artefactos del sistema. Se clasifican generalmente en dos categorías:

- **Upper-CASE (Herramientas de Planificación y Análisis):** Enfocadas en las fases iniciales del desarrollo. Permiten el levantamiento de requisitos, el análisis de viabilidad y la creación de diagramas de alto nivel (como diagramas de flujo de procesos o casos de uso).
- **Lower-CASE (Herramientas de Implementación y Mantenimiento):** Orientadas al desarrollo técnico. Permiten la generación automática de código a partir de modelos (ingeniería directa), la creación de esquemas de bases de datos y la documentación técnica final.

### **2. Uso de Software de Modelado (StarUML y Dia)**

Para el modelado sistémico, el uso de herramientas especializadas es indispensable para mantener la coherencia semántica. Dos ejemplos representativos con enfoques distintos son StarUML y Dia.

- **StarUML:** Es una herramienta de modelado de alto nivel diseñada específicamente para cumplir con el estándar UML (Unified Modeling Language).

**Funcionalidades:** Permite la creación de diagramas de clases, secuencias, actividades y casos de uso con validación semántica (asegura que las relaciones entre objetos sigan las reglas de UML).

**Ventaja técnica:** Su capacidad de "ingeniería directa e inversa" permite generar el esqueleto de código (ej. clases en Java, C# o Python) a partir de un diagrama de clases, o bien, importar código fuente existente para visualizar automáticamente su estructura en un diagrama. Es ideal para proyectos de mediana y gran escala donde la arquitectura del software es compleja.

- **Dia:** Es una herramienta de diagramación de propósito general, de código abierto, diseñada para ser ligera y versátil.

**Funcionalidades:** Utiliza un sistema de cuadrículas y formas predefinidas que pueden adaptarse para diversos tipos de esquemas, incluyendo diagramas de flujo, topologías de redes y diagramas UML básicos.

**Ventaja técnica:** Su sencillez. Al no tener la complejidad de un motor de validación de código como las herramientas CASE avanzadas, Dia es extremadamente útil para bosquejos rápidos, mapas mentales de procesos o documentación técnica que requiere agilidad por encima de la automatización estricta.

### 3. Beneficios de Implementar Herramientas CASE

El uso de este tipo de software aporta valor estratégico a cualquier proyecto de ingeniería por tres razones principales:

1. **Consistencia Arquitectónica:** Cuando un analista modifica un elemento en un diagrama, la herramienta CASE asegura que dicho cambio se replique o actualice en el resto de las vistas del sistema (por ejemplo, al actualizar una clase, se refleja en los diagramas de interacción asociados).
2. **Facilidad de Mantenimiento:** Un sistema documentado con herramientas de modelado es más sencillo de comprender para nuevos desarrolladores. La "memoria del sistema" reside en los modelos, lo que reduce la dependencia del conocimiento tácito de los programadores originales.
3. **Reducción de Errores de Diseño:** La capacidad de visualizar la estructura lógica antes de programar permite detectar redundancias, fallos de herencia o ciclos de dependencias circulares que, de otra forma, solo se descubrirían

durante la fase de codificación o pruebas, cuando el costo de corrección es mucho mayor.